PROJET DE CONCEPTION LOGICIELLE

Dungeon Composite

Un jeu de donjon construit sur quatre patrons de conception

Composite

Décorateur

État

Singleton

Auteur	[Votre nom]
Cours	[Cours / module]
Encadrant	[Nom de l'enseignant]
Date	[Date]

Langage : Java 17 - Interfaces : console (Scanner) et 2D (Swing)

1. Présentation du projet

Dungeon Composite est un petit jeu d'exploration de donjon. Le joueur descend **5 étages**. À chaque étage, il choisit **une salle** parmi deux ou trois, combat ou évite les monstres qui s'y trouvent, ramasse les trésors puis prend l'escalier. Certaines salles cachent un **caveau** gardé par un boss. Chaque action peut réveiller les monstres endormis, et les monstres éveillés contre-attaquent. Le but est d'atteindre le fond vivant avec le plus d'or possible.

Le même modèle de jeu est utilisé par deux interfaces interchangeables :

- **Version console** (`Main` / `GameManager.run()`) : menus numérotés lus avec un `Scanner`.
- **Version 2D** (`Game2D`, `Sprites`) : une fenêtre Swing avec des sprites en pixel art. Le héros se déplace avec les flèches du clavier, et marcher sur un élément permet de le combattre, de le ramasser ou d'y entrer.

Aucune des deux interfaces ne contient de règles du jeu. Elles communiquent avec le modèle uniquement à travers l'interface `DungeonElement` et le singleton `GameManager`, ce que les patrons de conception rendent possible.

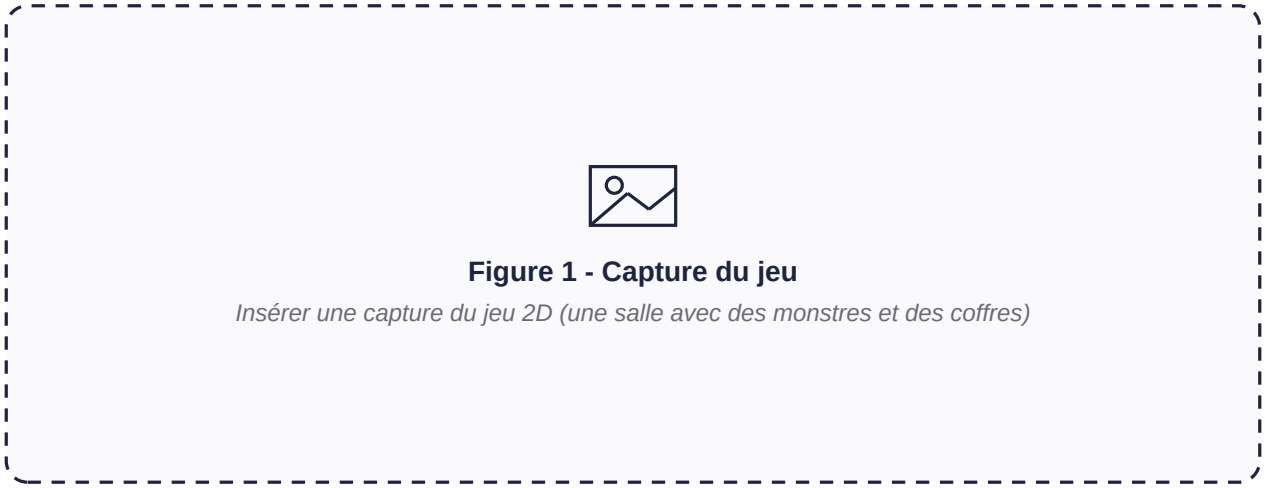


Figure 1 - Capture du jeu

Insérer une capture du jeu 2D (une salle avec des monstres et des coffres)

2. Les patrons en un coup d'oeil

Patron	Catégorie	Classes principales	Rôle dans le jeu
Composite	Structurel	DungeonElement, Room, Monster, Treasure	Donjon -> étages -> salles -> caveaux, tous traités de la même façon
Décorateur	Structurel	ElementDecorator, EliteMonster, BossMonster, CursedTreasure, EnchantedTreasure	Ajoute des modificateurs (x2, x3, malédiction, sceau) à n'importe quel élément à l'exécution
État	Comportemental	MonsterState, AwakeState, SleepingState, DefeatedState	Le comportement d'un monstre (dégâts, prime, libellé) change selon son état
Singleton	Créationnel	GameManager	Une seule partie : un donjon, un joueur, un journal de messages

3. Architecture

Le diagramme de classes ci-dessous montre comment les patrons s'articulent. `DungeonElement` est au centre de la conception : c'est le *Composant* du Composite et le type que chaque Décorateur enveloppe. `Monster` délègue son comportement à un `MonsterState`, et `GameManager` possède la `Room` racine.

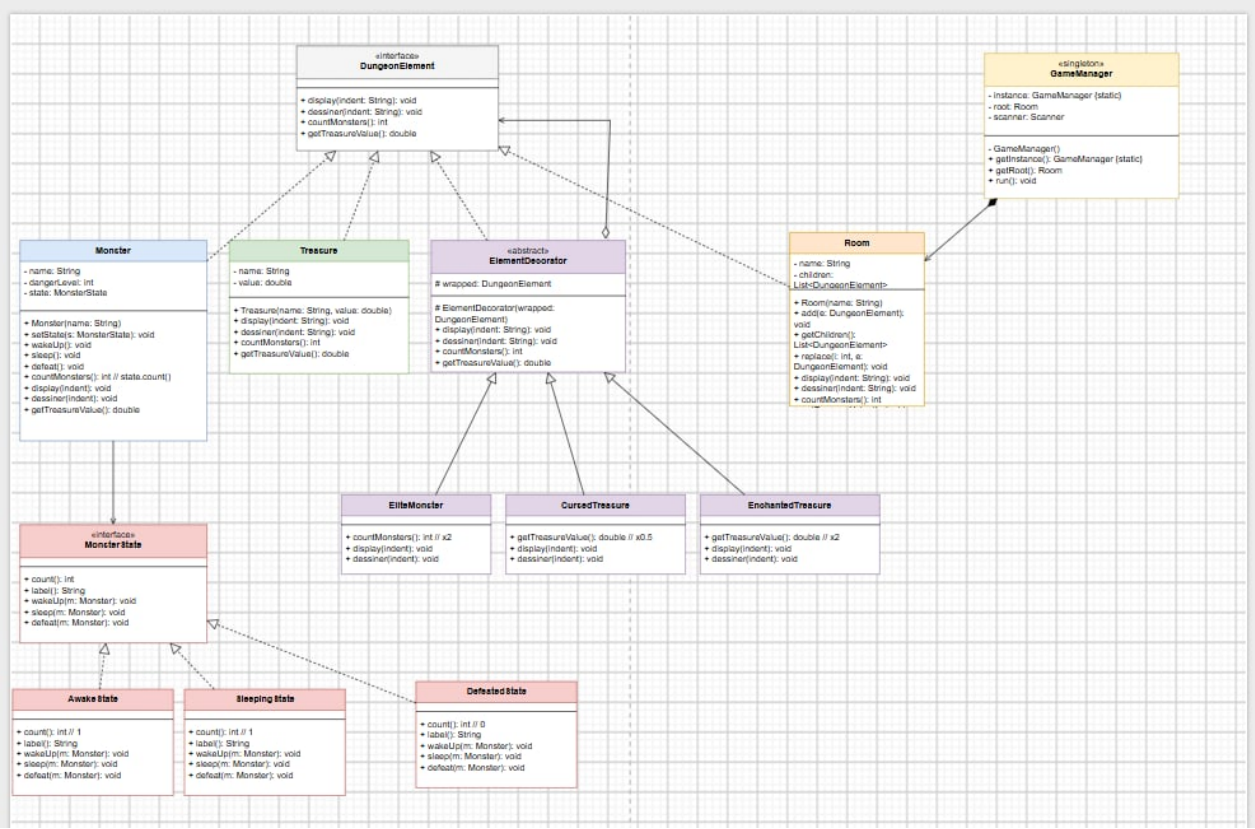


Figure 2 - Diagramme de classes UML du projet

Chaque élément répond aux mêmes questions :

Méthode	Signification dans le jeu
describe()	Texte affiché au joueur ("Elite Goblin (danger 2, sleeping)")
countMonsters()	Nombre de monstres vivants contenus dans l'élément
getTreasureValue()	Or encore récupérable (prime pour les monstres)
threat()	Dégâts infligés à l'instant à quiconque le touche
collect(player)	Retire l'or et le renvoie (bat les monstres)
wakeUp() / sleep()	Propagation du bruit

PATRON 1

Composite

Composer des objets en arborescence et permettre de traiter de la même façon un objet seul et un groupe d'objets.

Problème

Un donjon est imbriqué : le donjon contient des étages, les étages des salles, les salles des monstres, des trésors et parfois un caveau, qui est lui-même une salle. Le jeu a sans cesse besoin de totaux comme « combien d'or y a-t-il dans cette salle, caveau compris ? » ou « combien de monstres sont encore en vie ? ». Sans type commun, chaque boucle aurait besoin de tests `instanceof` et de cas particuliers à chaque niveau.

Solution dans le projet

Rôle	Classe	Responsabilité
Composant	DungeonElement	Interface commune à tout ce qui se trouve dans le donjon
Composite	Room	Contient une List<DungeonElement> et délègue chaque appel à ses enfants
Feuille	Monster	Un ennemi ; ses valeurs dépendent de son état
Feuille	Treasure	Un tas d'or qui se vide une fois ramassé

```
public class Room implements DungeonElement {
    private final List<DungeonElement> children = new ArrayList<>();

    public double getTreasureValue() {
        return children.stream().mapToDouble(DungeonElement::getTreasureValue).sum();
    }
    public int countMonsters() {
        return children.stream().mapToInt(DungeonElement::countMonsters).sum();
    }
    public void wakeUp() { children.forEach(DungeonElement::wakeUp); }
}
```

Dans le jeu

- Tout le donjon forme **un seul arbre** : `Dungeon -> Floor n -> Room nX -> Vault`, construit dans `buildDungeon()`.
- Un total ne demande qu'un appel : `root.getTreasureValue()` donne l'or de tout le donjon pour le score final.
- Un caveau n'est qu'une Room dans une Room. En 2D, y entrer l'empile sur une pile, et le même code de dessin l'affiche.
- `display(indent)` affiche de façon récursive le bilan « ce qui reste » à la fin de la version console.



Figure 3a - Jeu

Une salle contenant la porte d'un caveau



Figure 3b - Code

Room.java ou buildDungeon()

PATRON 2

Décorateur

Ajouter dynamiquement des responsabilités à un objet, sans passer par l'héritage.

Problème

Monstres et trésors existent en plusieurs variantes : élite, boss, maudit, enchanté. Créer une sous-classe pour chaque combinaison (EliteGoblin, CursedGem, BossEliteOrc...) ferait exploser le nombre de classes. De plus, les variantes doivent être choisies *au hasard lors de la génération*, et non fixées à la compilation.

Solution dans le projet

`ElementDecorator` implémente `DungeonElement`, contient un `DungeonElement` enveloppé et lui transmet chaque appel. Les décorateurs concrets ne redéfinissent que ce qu'ils modifient :

Décorateur	Modifications	Effet pour le joueur
EliteMonster	countMonsters x2, threat x2, prime x2	Frappe plus fort, double récompense. 20 % des monstres
BossMonster	threat x3, prime x2, préfixe "BOSS"	Gardien du caveau, l'ennemi le plus dangereux
CursedTreasure	après collect(), appelle player.curse()	Tout l'or suivant est divisé par deux. Un pari risqué
EnchantedTreasure	valeur x2, mais 0 tant que le boss vit	Relique du caveau, scellée jusqu'à la mort du boss

```
public class EnchantedTreasure extends ElementDecorator {
    private final DungeonElement boss;
    private boolean sealed() { return boss.countMonsters() > 0; }
    @Override
    public double collect(Player player) {
        return sealed() ? 0 : wrapped.collect(player) * 2;
    }
}
// génération : les décorateurs sont appliqués au hasard, à l'exécution
room.add(random.nextDouble() < 0.2 ? new EliteMonster(m) : m);
```

Dans le jeu

- Le code du jeu ne sait jamais si un élément est décoré. `interact()` appelle `threat()` et `collect()`, et les décorateurs ajustent le résultat.
- Les décorateurs se cumulent : `new BossMonster(new EliteMonster(m))` donnerait une menace x6 sans aucune nouvelle classe.
- La vue 2D rend les décorations visibles : halo doré pour Elite, cornes pour Boss, coffre violet pour Cursed, chaînes sur un coffre enchanté scellé.



Figure 4a - Jeu

Un boss dans un caveau, ou un coffre maudit



Figure 4b - Code

ElementDecorator + une sous-classe

PATRON 3

État

Permettre à un objet de changer de comportement quand son état interne change, comme s'il changeait de classe.

Problème

Un monstre se comporte très différemment selon qu'il dort, est éveillé ou est mort : dégâts infligés, s'il compte comme vivant, prime rapportée, réaction au bruit. Écrire `if (state == AWAKE) ...` dans chaque méthode disperse les règles.

Solution dans le projet

État	count()	attack()	wakeUp()	defeat()
SleepingState	1	0	-> Awake	-> Defeated
AwakeState	1	danger x 4	inchangé	-> Defeated
DefeatedState	0	0	inchangé	inchangé

```
public class Monster implements DungeonElement {
    private MonsterState state;
    public int threat()        { return state.attack(this); }
    public int countMonsters() { return state.count(); }
    public void wakeUp()       { state.wakeUp(this); }
}
public class SleepingState implements MonsterState {
    public int attack(Monster m) { return 0; }
    public void wakeUp(Monster m) { m.setState(new AwakeState()); }
}
```

Dans le jeu

- **Discrétion** : les monstres endormis n'infligent aucun dégât, le joueur peut donc piller autour d'eux.
- **Bruit** : après chaque action, `makeNoise()` a 35 % de chances d'appeler `wakeUp()` sur chaque élément. C'est l'état qui décide si cela change quelque chose.
- **Riposte** : `strikeBack()` additionne les `threat()`, et seuls les monstres éveillés y contribuent.
- **Défaite** : `DefeatedState.count() = 0` descelle automatiquement les reliques des caveaux (EnchantedTreasure).
- **Visuels 2D** selon l'état : yeux rouges et sautillement (éveillé), yeux fermés et « zZ » (endormi), crâne (vaincu).



Figure 5a - Jeu

Monstre endormi, monstre éveillé et un crâne



Figure 5b - Code

MonsterState et ses 3 états

PATRON 4

Singleton

Garantir qu'une classe n'a qu'une seule instance et fournir un point d'accès global à celle-ci.

Problème

Une partie comporte exactement un donjon, un joueur et un journal de messages. Les deux interfaces (console et 2D) doivent travailler sur cette même partie. Si chacune créait son propre gestionnaire, on pourrait se retrouver avec deux donjons ou deux joueurs différents.

Solution dans le projet

```
public final class GameManager {
    private static GameManager instance;
    private final Player player = new Player();
    private final Room root;

    private GameManager() { root = buildDungeon(); }   // constructeur privé

    public static GameManager getInstance() {
        if (instance == null) instance = new GameManager();
        return instance;
    }
}

// les deux interfaces
GameManager.getInstance().run();                       // Main (console)
private final GameManager game = GameManager.getInstance();  // Game2D
```

Dans le jeu

- Le donjon est généré **une seule fois**, dans le constructeur privé, au premier accès.
- Les règles du jeu sont centralisées : `interact()`, `strikeBack()` et `makeNoise()` sont partagées par la version console et la version 2D.
- La fenêtre 2D lit le joueur, la racine du donjon et le journal via `getPlayer()`, `getRoot()` et `takeLog()`.

Limite. L'initialisation paresseuse n'est pas thread-safe. Ce n'est pas un problème ici, car le jeu tourne sur un seul thread (celui de la console ou celui des événements Swing). Une version multi-thread utiliserait une instance créée d'avance ou une classe holder.



Figure 6a - Jeu

Barre d'état : PV, or, journal de messages



Figure 6b - Code

GameManager.getInstance()

5. Comment les patrons collaborent

Un seul déplacement dans le jeu 2D fait intervenir les quatre patrons. Supposons que le héros fonce sur un **Elite Goblin éveillé**, dans une salle qui contient aussi un caveau :

#	Ce qui se passe	Patron
1	Game2D demande à `GameManager.getInstance()` d'exécuter `interact(target, items)`	Singleton
2	`target.threat()` : EliteMonster double ce que renvoie le Monster enveloppé	Décorateur
3	Monster délègue à `AwakeState.attack()` = danger x 4	État
4	`collect()` verse la prime (x2 grâce au décorateur) et passe l'état à Defeated	État + Décorateur
5	`strikeBack()` additionne les `threat()` de la salle. Les caveaux sont ignorés car ce sont des Room	Composite
6	`makeNoise()` appelle `wakeUp()`. Un monstre endormi change d'état, un mort l'ignore	État
7	La vue se redessine en interrogeant chaque élément : `countMonsters()`, `threat()`, `getTreasureValue()`	Composite

Bénéfices constatés

- **Ouvert à l'extension** : un nouveau modificateur (ex. *PoisonedTreasure*) tient en une nouvelle classe décorateur, sans rien changer d'autre.
- **Un nouvel état de monstre** (ex. *Enraged*) est un nouveau MonsterState, sans modifier Monster.
- **Deux interfaces, un seul modèle** : la version 2D a été ajoutée sans toucher à aucune classe des patrons.
- **Aucun test de type pour les règles du jeu** : totaux et dégâts viennent d'appels polymorphes sur l'arbre.



Figure 7 - Écran de fin

Insérer une capture de « YOU ESCAPED! » ou « YOU DIED » avec l'or final

6. Contenu complémentaire

Espace réservé au contenu que vous ajouterez : analyse personnelle, difficultés rencontrées, résultats de tests, améliorations possibles, etc.

6.1 [Titre de la section]

[Votre texte ici]



Figure 8 - Capture d'écran

Capture du jeu ou du code illustrant cette section

6.2 [Titre de la section]

[Votre texte ici]



Figure 9 - Capture d'écran

Capture du jeu ou du code illustrant cette section

7. Conclusion

[Votre conclusion ici]